MongoDB 8.0 · INDICES TEORIA

Indexación

Teoría del capítulo 4

De B-tree y explain a índices compuestos, multikey, texto, geoespaciales, hashed, unique y TTL.

Ejemplos del curso: los comandos se explican para una base de datos de laboratorio preparada con MongoDB Community 8.0. Consulta las prácticas del capítulo para ejecutar el código y comprobar los resultados. Antes de interpretar un ejemplo, revisa qué datos, índices o usuarios requiere.

Itinerario de aprendizaje

Del síntoma de lentitud a una decisión de diseño

Qué debe saber hacer el alumno

- Explicar el coste de un escaneo frente a un acceso por índice con evidencia de ejecución.
- Diseñar índices simples y compuestos, justificar ESR y comprender sus excepciones.
- Distinguir multikey, texto, geoespacial, hashed, unique y TTL sin atribuirles capacidades inexistentes.

Cómo se lee este manual

Los 38 capítulos cortos son unidades independientes, pero siguen una secuencia que parte del problema, interpreta planes, diseña índices y termina en mantenimiento. Los comandos están enlazados con el cuaderno de prácticas. No es necesario impartir cada página en las tres horas: los apartados avanzados son material autónomo.

Una única colección de referencia

Trabajaremos con ventas sintéticas: numero, clienteId, estado, fecha, importe, etiquetas, descripcion, idGrupo y ubicacion. La carga básica incorpora 200.000 documentos y puede elevarse a 1.000.000. Los valores son deterministas para facilitar predicciones, pero los tiempos no lo son.

FILTRO → PLAN DE ACCESO → CONTADORES → DECISIÓN

Cómo se demuestra una mejora

Conserva el predicado y volumen y compara plan ganador, nReturned, totalKeysExamined y totalDocsExamined. Considera tiempos y coste de mantenimiento como datos complementarios. No supongas que un IXSCAN es siempre mejor ni que una consulta más corta es necesariamente óptima.

Fuentes de referencia

MongoDB 8.0: https://www.mongodb.com/docs/v8.0/core/indexes/index-types/

Explain: https://www.mongodb.com/docs/manual/reference/explain-results/

TTL: https://www.mongodb.com/docs/manual/core/index-ttl/

> Los prefijos compuestos, ESR, el tamaño y mantenimiento de índices, las consultas cubiertas y `explain()` se estudian mediante ejemplos progresivos en la teoría y las prácticas.

01. El problema de buscar sin índices

Explicación, evidencia y criterio técnico

Concepto y mecanismo

COLLSCAN examina candidatos recorriendo la colección, aunque la consulta devuelva pocos resultados. La selectividad indica la fracción de filas que supera el filtro.

Piensa en el registro de una tienda: pedir una factura por numero no debería obligar a abrir cada carpeta. Si existen 200.000 ventas y el filtro devuelve una, la lectura secuencial puede examinar del orden de 200.000 documentos. Esa desproporción es más informativa que un tiempo aislado. El plan puede incluir otras etapas; hay que mirar el árbol entero. Para obtener una línea de base limpia ejecuta la medición ANTES de crear índices secundarios y conserva el tamaño de la colección. Una vez instalado un índice específico, repite exactamente el mismo filtro, proyección y límite.

Caso que resuelve

COLECCIÓN → COLLSCAN → RESULTADO

Una venta concreta entre un millón no debería obligar a leer todo el conjunto. Antes de diseñar índices se enumeran consultas reales, sus filtros, proyecciones y órdenes.

Demostración o comprobación

```
db.ventas.find({numero:250001}).explain("executionStats")
```

Segunda situación para contrastar

```
db.ventas.find({numero:25001}).limit(1).explain("executionStats")
```

La presencia de limit(1) no elimina el recorrido previo de documentos hasta localizar la operación. Compara el número examinado con una consulta por _id y razona qué estructura necesitaría numero.

Cómo interpretar el resultado

Con solo *id* disponible, el filtro numero o clienteId no dispone de acceso secundario selectivo; normalmente los documentos examinados se acercarán al tamaño completo.

> **Precaución técnica.** La colección muy pequeña puede rendir mejor con un escaneo; no se indexa por reflejo.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Una venta concreta entre un millón no debería obligar a leer todo el conjunto.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

02. Qué guarda un índice B-tree

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice ordena claves y referencias a documentos; no es una copia íntegra de cada documento. Las claves mantienen un orden aprovechable para igualdad, rangos y ciertas ordenaciones.

La estructura del índice mantiene pares ordenados de clave y localizador; cuando hay claves repetidas, las entradas comparten tramos vecinos. El motor navega hasta la primera coincidencia y recorre las siguientes sin inspeccionar documentos de otros clientes. Eso no significa que guarde toda la venta: para recuperar importe, fecha y descripcion puede necesitar un FETCH adicional. Dibujar el árbol como directorio de accesos es válido; dibujarlo como una copia completa de la colección no lo es.

Caso que resuelve

CLAVE → B-TREE → DOCUMENTO

En {clienteId:1}, las entradas del mismo cliente quedan próximas. El beneficio aparece al evitar leer documentos no relevantes.

Demostración o comprobación

```
db.ventas.createIndex({clienteId:1},{name:"ix_cliente"})
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809},{_id:0,clienteId:1}).explain("executionStats")
```

No hace falta devolver todos los campos de la venta. Con el índice de cliente y una proyección compatible se puede reducir la lectura de documentos; la estructura ordenada separa localizar claves y obtener documentos.

Cómo interpretar el resultado

La creación modifica el catálogo de índices, no la forma lógica de los documentos. Las operaciones de igualdad recorren claves contiguas y recuperan solo candidatos.

> **Precaución técnica.** El índice ocupa memoria y disco y aumenta trabajo de inserción.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «En {clienteId:1}, las entradas del mismo cliente quedan próximas.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

03. Selectividad y cardinalidad

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Cardinalidad es el número de valores distintos; selectividad describe cuán discriminante es un predicado. Una columna de cuatro estados suele filtrar peor que un número de operación único.

En el generador, estado tiene cuatro valores y clienteId recorre 20.000 valores. Para 200.000 ventas caben aproximadamente 50.000 documentos en el primer filtro y diez en el segundo, porque la distribución es determinista. Un índice sobre estado no es necesariamente inútil: puede aportar la ordenación de una consulta compuesta o reducir el trabajo si solo se proyectan campos indexados. Para elegir, compara consultas reales, no etiquetas simplistas como «alta cardinalidad = bueno».

Caso que resuelve

estado="pagada" puede coincidir con un cuarto de la colección. Se priorizan accesos frecuentes y selectivos, sin concluir que toda baja cardinalidad sea inútil.

Demostración o comprobación

```
db.ventas.distinct("estado")
```

Segunda situación para contrastar

```
db.ventas.countDocuments({estado:"pagada"})
db.ventas.countDocuments({clienteId:809})
```

La diferencia de órdenes de magnitud entre las dos cuentas muestra qué predicado discrimina más. Repite con un volumen distinto: las proporciones siguen siendo sintéticas y no describen necesariamente una empresa.

Cómo interpretar el resultado

Con estado de cuatro valores puede ser razonable un escaneo extenso; con numero casi único, leer un tramo corto tiene una ventaja más clara.

> **Precaución técnica.** El conjunto de filtros y la ordenación cambian la utilidad efectiva.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «estado="pagada" puede coincidir con un cuarto de la colección.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

04. Costes de lectura y escritura

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Cada índice mejora potencialmente lecturas pero exige mantener entradas en insert, update y delete. Conviene medir en la misma carga de trabajo.

No basta con celebrar que la búsqueda bajó de miles de documentos a unas decenas. En el otro lado, cada inserción modifica tantas estructuras auxiliares como correspondan. Si se actualiza un campo indexado, también habrá cambios en las entradas. Un sistema orientado a ingestión masiva y uno orientado a búsquedas interactivas necesitan estrategias diferentes. Al medir, realiza primero una carga controlada sin índices y después el mismo tamaño en una colección equivalente con índices, evitando concluir por una sola ejecución.

Caso que resuelve

Una aplicación que inserta miles de ventas cada segundo paga por cada índice agregado. Diseñar el mínimo conjunto de índices que cubra consultas prioritarias.

Demostración o comprobación

```
db.ventas.getIndexes()
```

Segunda situación para contrastar

```
db.ventas.getIndexes().map(x=>({name:x.name,key:x.key}))
```

Enumera índices antes de medir el coste de inserción. Un resultado rápido de find no informa sobre el impacto de crear cinco estructuras adicionales en la tasa de escrituras.

Cómo interpretar el resultado

Un INSERT debe actualizar tantas estructuras como índices corresponda; medir solo una consulta de lectura oculta este coste.

> **Precaución técnica.** No prometer aceleración de escritura tras crear índices adicionales.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Una aplicación que inserta miles de ventas cada segundo paga por cada índice agregado.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

05. Crear un índice simple

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice simple incluye un solo campo. Es una puerta de entrada excelente para enseñar búsquedas por igualdad y diferencias con COLLSCAN.

La clave clienteId identifica a un comprador, no una venta. En 200.000 filas sintéticas cada comprador aparece unas diez veces; por ello no debe crearse como unique. Tras crear ix_cliente, una búsqueda por clienteId debería inspeccionar una fracción pequeña, aunque siga leyendo documentos para devolver el resto de campos. Verifica además que el índice aparece con el nombre correcto en getIndexes() y que se utiliza con el filtro planteado, no con un filtro ajeno.

Caso que resuelve

Encontrar todas las ventas de clienteId 809. Comparar documentos examinados antes y después; repetir la consulta idéntica.

Demostración o comprobación

```
db.ventas.createIndex({clienteId:1},{name:"ix_cliente"})
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).limit(5).explain("executionStats")
```

No uses una query por fecha para comprobar un índice que solo contiene clienteId. Debe existir correspondencia exacta entre la clave creada y los predicados que quieres estudiar.

Cómo interpretar el resultado

Tras crear ix_cliente, el filtro por clienteId debería reducir notablemente totalDocsExamined; el índice no cubre campos de proyección ajenos a su clave.

> **Precaución técnica.** La consulta debe seleccionar suficientes datos para mostrar diferencia observable.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Encontrar todas las ventas de clienteId 809.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

06. Índices ascendente y descendente

Explicación, evidencia y criterio técnico

Concepto y mecanismo

En un índice simple MongoDB puede recorrer ambas direcciones. En índices compuestos la combinación de direcciones sí afecta a ciertas ordenaciones.

Para una sola clave, MongoDB puede recorrer el mismo árbol hacia delante o atrás. Crear simultáneamente {fecha:1} y {fecha:-1} no suele aportar dos capacidades diferentes. Con varias claves sí existe una restricción sobre el patrón completo de la ordenación: el motor puede recorrer el índice en su dirección o en la dirección inversa total, pero no invertir arbitrariamente cada columna. Pide al alumno que compare ordenación de cliente y fecha, no únicamente fechas aisladas.

Caso que resuelve

Listar fechas descendentes dentro de un mismo cliente. Escoger orientación del compuesto a partir de sorts de la carga real.

Demostración o comprobación

```
db.ventas.createIndex({fecha:-1},{name:"ix_fecha_desc"})
```

Segunda situación para contrastar

```
db.ventas.find().sort({fecha:1}).limit(5).explain("queryPlanner")
```

Si hay un índice simple en fecha, puede recorrerse en sentido ascendente o descendente. A continuación compara el caso de dos campos, donde las combinaciones no son arbitrarias.

Cómo interpretar el resultado

Un índice simple puede recorrerse en orden inverso; su orientación aislada no suele justificar mantener dos índices redundantes.

> **Precaución técnica.** No crear {campo:1} y {campo:-1} sin justificar el coste.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Listar fechas descendentes dentro de un mismo cliente.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

07. El índice de _id

Explicación, evidencia y criterio técnico

Concepto y mecanismo

MongoDB crea automáticamente un índice único por _id en colecciones ordinarias. Es parte de la identidad de cada documento.

Cada colección normal contiene un índice único sobre _id que soporta las búsquedas por identificador del documento. El campo numero no se convierte en _id automáticamente: aunque represente una clave de negocio, es otro camino de acceso. El nombre de la clave funcional importa también por integridad: si numero debe ser irrepetible, un índice unique en numero representa una regla distinta. No elimines ni modifiques _id para demostrar la diferencia.

Caso que resuelve

getById de una operación administrativa. Aprovechar _id para lecturas y actualizaciones identificadas.

Demostración o comprobación

```
db.ventas.getIndexes().filter(x=>x.name==="_id_")
```

Segunda situación para contrastar

```
db.ventas.findOne({_id:db.ventas.findOne()._id})
```

El acceso por _id sirve para identificar un documento; numero es una propiedad distinta. Si la aplicación recibe codigo de negocio, necesitará elegir una clave de acceso apropiada.

Cómo interpretar el resultado

getIndexes debe contener *id* con unique; verificar que buscar por numero carece de privilegio automático por el mero parecido semántico.

> **Precaución técnica.** Un índice de _id no resuelve búsquedas por otros campos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «getById de una operación administrativa.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

08. Compuestos: el prefijo

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice compuesto admite eficientemente consultas por su prefijo inicial. El índice {clienteId,fecha} puede apoyar {clienteId}, pero no ofrece igual acceso a {fecha} aislada.

Considera el compuesto {clienteId:1,fecha:-1}: primero agrupa ventas por comprador y solo dentro de cada comprador ordena por fecha reciente. El índice facilita tanto la igualdad por comprador como una consulta que añada fecha, pero no dispone de una ordenación global por fecha que prescinda de clienteId. Es la regla de prefijo: para seleccionar entradas con eficiencia se necesita comenzar por la primera parte de la clave. Distínguela del simple hecho de que una consulta pueda forzar un recorrido de ese índice.

Caso que resuelve

CLIENTE → FECHA → PREFIJO

Buscador por cliente y después intervalos de fecha. Ordenar campos atendiendo a patrones de consulta, no al alfabeto.

Demostración o comprobación

```
db.ventas.createIndex({clienteId:1,fecha:-1})
```

Segunda situación para contrastar

```
db.ventas.find({fecha:{$gte:ISODate("2025-01-01")}}).limit(10).explain("executionStats")
```

La condición anterior no contiene clienteId. Tener fecha como segundo elemento de una clave compuesta no equivale a poseer un índice global que comienza por fecha.

Cómo interpretar el resultado

El prefijo clienteId delimita la región útil del índice; una consulta solo por fecha no aprovecha el mismo prefijo delimitador.

> **Precaución técnica.** La utilidad depende también del plan, de la ordenación y de la selectividad.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Buscador por cliente y después intervalos de fecha.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

09. Compuestos: orden de campos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Dos índices con los mismos campos en distinto orden no son intercambiables. Su estructura ordenada induce accesos diferentes.

En {estado:1,fecha:-1}, buscar pagadas del último mes delimita primero un conjunto por estado; en {fecha:-1,estado:1}, pedir «ventas recientes» puede resultar más directo sin filtrar por estado. Son índices distintos con costes diferentes. No deduzcas el orden óptimo de los campos solamente de su cardinalidad: una ordenación solicitada, un limit y la frecuencia del patrón pueden cambiar la decisión. Trabaja con dos consultas: «pagadas recientes» y «recientes de cualquier estado».

Caso que resuelve

Comparar {estado,fecha} frente a {fecha,estado}. Planificar prefijos de igualdad y sorts relevantes.

Demostración o comprobación

```
db.ventas.createIndex({estado:1,fecha:-1},{name:"ix_estado_fecha"})
```

Segunda situación para contrastar

```
db.ventas.find({estado:"pagada"}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Este patrón favorece {estado:1,fecha:-1}. Ahora elimina el estado del filtro y observa si la ordenación se sigue resolviendo sin SORT: es otro contrato de consulta.

Cómo interpretar el resultado

La pareja estado-fecha prioriza consultas por estado y tiempo; fecha-estado prioriza otros accesos y no garantiza ordenación equivalente.

> **Precaución técnica.** No duplicar ambos si las consultas no lo justifican.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Comparar {estado,fecha} frente a {fecha,estado}.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

10. Estrategia ESR

Explicación, evidencia y criterio técnico

Concepto y mecanismo

ESR recomienda colocar Equalities, después Sort y después Range en muchos índices compuestos. No es una ley universal: la selectividad de rangos puede justificar ERS.

ESR significa Igualdad, Sort, Rango. La clave {estado:1,fecha:-1,importe:1} puede satisfacer estado=pagada, producir fechas ordenadas y evaluar importe. Pero si importe>=890 deja muy pocos resultados, situar importe antes de fecha puede leer muchas menos claves aun a costa de una fase SORT. Ese es el motivo por el que ESR es un punto de partida y no un dogma. El informe debe mostrar keysExamined, docsExamined y la presencia de SORT.

Caso que resuelve

EQUALITY → SORT → RANGE

Estado pagada + ordenar fecha + importe mayor de 600. Verificar la estrategia con explain, no con intuición.

Demostración o comprobación

```
db.ventas.createIndex({estado:1,fecha:-1,importe:1},{name:"ix_esr"})
```

Segunda situación para contrastar

```
db.ventas.find({estado:"pagada",importe:{$gte:890}}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Con rango muy selectivo, una clave estado-importe-fecha puede encontrar pocos registros; con estado-fecha-importe se preserva otra ventaja. La respuesta depende de las estadísticas y de cuántos resultados quieres devolver.

Cómo interpretar el resultado

Si fecha satisface sort y importe es rango posterior, es posible ahorrar SORT aunque se examinen más claves; un ERS selectivo puede resultar mejor.

> **Precaución técnica.** La presencia de varios rangos, sorts o filtros modifica el resultado.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Estado pagada + ordenar fecha + importe mayor de 600.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

11. Igualdad y ordenación

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Las igualdades en un prefijo delimitan un tramo y las siguientes claves pueden proveer orden sin SORT en memoria.

La igualdad en clienteId convierte un orden compuesto {clienteId:1,fecha:-1} en una secuencia cronológica ya utilizable. Sin igualdad, las fechas estarían ordenadas dentro de muchos grupos de clientes y una solicitud global de últimas ventas no disfrutaría del mismo orden. Añadir limit(10) permite detener el recorrido cuando hay suficiente información, pero no transforma un índice inadecuado en adecuado. Comprueba el sortPattern y las etapas del plan según el motor.

Caso que resuelve

Filtrar por cliente y ordenar por fecha descendente. Observar si el plan introduce SORT o utiliza el orden del índice.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).sort({fecha:-1}).limit(15)
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).sort({fecha:1}).limit(5).explain("queryPlanner")
```

La inversión total de recorrido sobre un índice cliente+fecha permite otra dirección de tiempo para ese único cliente. Requiere entender dirección de recorrido frente a dirección física declarada.

Cómo interpretar el resultado

Con igualdad por cliente, las fechas quedan ordenadas dentro de ese bloque; sin igualdad el orden global de fechas no es necesariamente el deseado.

> **Precaución técnica.** La dirección en los compuestos importa cuando existen varios campos ordenados.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Filtrar por cliente y ordenar por fecha descendente.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

12. Igualdad y rango

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Una condición de rango puede leer un tramo contiguo cuando sigue campos de igualdad.

Un rango acota un intervalo: importe entre 650 y 660, por ejemplo. Si el índice empieza por un campo de igualdad y continúa por importe, el motor puede navegar hasta el límite inferior y parar en el superior. En cambio, si la condición selecciona casi todas las ventas, un COLLSCAN puede resultar competitivo. El objeto indexBounds del plan, cuando aparece, ayuda a ver qué intervalo se leyó. No confundas «el índice existe» con «los límites se aprovecharon».

Caso que resuelve

Ventas pagadas de importe superior a 600. Inspeccionar indexBounds y los documentos examinados.

Demostración o comprobación

```
db.ventas.find({estado:"pagada",importe:{$gt:600}})
```

Segunda situación para contrastar

```
db.ventas.find({importe:{$gte:650,$lt:660}}).explain("executionStats")
```

Un intervalo corto delimita pocos valores. Repite con {$gte:5,$lt:905}: el segundo intervalo contiene casi toda la distribución y la justificación del índice cambia.

Cómo interpretar el resultado

El límite inferior o superior de importe aparece en indexBounds si existe clave aprovechable; los documentos devueltos no bastan para medir eficiencia.

> **Precaución técnica.** Un rango muy amplio reduce la ventaja del índice.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Ventas pagadas de importe superior a 600.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

13. Sort y límites

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice adecuado puede evitar una fase de ordenación costosa; un limit reduce el volumen devuelto, pero no corrige por sí solo la búsqueda ineficiente.

La consulta «10 pagadas más recientes» combina tres costes: encontrar pagadas, ordenar las coincidencias y devolver diez. Con un compuesto que preserve el orden se puede leer solo el tramo útil; sin él, el motor puede trabajar sobre muchos candidatos antes de entregar diez. Dibuja dos flujos: FILTRO → SORT → LIMIT frente a ACCESO INDEXADO ORDENADO → LIMIT. Una mejora de latencia no prueba por sí sola que se evitó ordenar: hay que comprobar el plan.

Caso que resuelve

Top 10 de ventas más recientes de un estado. Medir plan antes de atribuir la mejora a limit.

Demostración o comprobación

```
db.ventas.find({estado:"pagada"}).sort({fecha:-1}).limit(10)
```

Segunda situación para contrastar

```
db.ventas.find({estado:"pagada"}).sort({fecha:-1}).limit(1).explain("executionStats")
```

El top 1 es un buen caso para separar ordenación indexada de ordenar un conjunto enorme y después descartar filas. Identifica si la etapa SORT aparece.

Cómo interpretar el resultado

Una respuesta de diez resultados puede requerir ordenar cientos de miles de candidatos; el árbol revela si SORT persiste.

> **Precaución técnica.** Un orden incorrecto del compuesto puede hacer que aparezca SORT.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Top 10 de ventas más recientes de un estado.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

14. Covered queries

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Si filtro y proyección quedan resueltos por el índice, el motor puede evitar FETCH. La exclusión explícita de _id suele ser relevante.

Una consulta cubierta exige que el índice proporcione tanto la condición como todos los campos devueltos. Con {clienteId:1,fecha:-1}, proyectar exclusivamente clienteId y fecha, excluyendo _id, puede resolverse sin leer documentos. Si agregas descripcion, aparece la necesidad de FETCH. El indicador decisivo es totalDocsExamined=0, junto con un plan compatible; nReturned puede ser mayor que cero. Una consulta cubierta no implica que solo se examinen las claves necesarias para los resultados.

Caso que resuelve

FILTRO → PROYECCIÓN → SIN FETCH

Consultar solo clienteId y fecha. Buscar totalDocsExamined=0 cuando se cumple la cobertura.

Demostración o comprobación

```
db.ventas.find({clienteId:809},{_id:0,clienteId:1,fecha:1})
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809},{_id:0,clienteId:1,fecha:1}).explain("executionStats")
```

En una consulta cubierta la lectura de documentos puede quedar en cero. Añade descripcion a la proyección y contrasta con el caso anterior sin cambiar filtro.

Cómo interpretar el resultado

Excluir _id y devolver exclusivamente clienteId y fecha permite, si se elige el compuesto, totalDocsExamined=0; agregar descripcion rompe la cobertura.

> **Precaución técnica.** No dar por cubierta una proyección que incorpora campos ausentes del índice.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Consultar solo clienteId y fecha.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

15. COLLSCAN, IXSCAN y FETCH

Explicación, evidencia y criterio técnico

Concepto y mecanismo

COLLSCAN lee documentos de colección; IXSCAN recorre entradas de índice; FETCH recupera documentos necesarios. Los nombres de etapas pueden variar con el motor.

COLLSCAN y IXSCAN nombran recorridos; FETCH señala recuperación de documentos. En explain puede verse FETCH encima de IXSCAN: significa que el índice orienta el acceso y los documentos se leen para terminar de resolver filtro o proyección. Un plan no es una cadena plana: puede haber ramas y etapas diferentes, especialmente con el motor de ejecución SBE de versiones modernas. Enseña a localizar indexName y los contadores, no a memorizar una captura específica de otro lanzamiento.

Caso que resuelve

IXSCAN → FETCH → RESULTADO

La misma consulta puede pasar de COLLSCAN a IXSCAN y FETCH. Leer el árbol ganador completo, no solo la primera etiqueta.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).explain("executionStats")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809,estado:"pagada"}).explain("executionStats")
```

La presencia de IXSCAN no implica que el filtro estado esté también resuelto por el mismo índice. Un FETCH puede aplicar parte del filtro sobre documentos recuperados.

Cómo interpretar el resultado

En un plan INDEX_SCAN+FETCH, las claves examinadas no deben confundirse con documentos leídos: ambas cantidades contestan preguntas distintas.

> **Precaución técnica.** No interpretar IXSCAN como prueba automática de una consulta óptima.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «La misma consulta puede pasar de COLLSCAN a IXSCAN y FETCH.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

16. Estadísticas que importan

Explicación, evidencia y criterio técnico

Concepto y mecanismo

nReturned, totalDocsExamined y totalKeysExamined permiten razonar sobre trabajo realizado. Un cociente elevado entre examinados y devueltos sugiere revisión.

nReturned cuenta resultados; totalDocsExamined cuenta exámenes de documentos; totalKeysExamined cuenta entradas de índices leídas. Si salen diez ventas y el motor examinó 200.000 documentos, hay potencial de optimización. Si examina solo diez claves pero diez documentos, la consulta es selectiva pero no cubierta. Si examina cien claves y cero documentos, puede estar cubierta pero seguir recorriendo claves adicionales. Cada contador describe un fenómeno distinto; comparar índices requiere leerlos juntos.

Caso que resuelve

KEYS → DOCS → RETURNED

20 documentos devueltos frente a 200000 examinados. Conservar capturas/JSON de antes y después bajo idénticos datos.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).explain("executionStats").executionStats
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).explain("executionStats").executionStats
```

Si nReturned es diez y docsExamined también diez, el acceso ya evita casi todo el escaneo. Un keysExamined de diez tampoco garantiza que el coste total de la petición sea cero.

Cómo interpretar el resultado

nReturned bajo con docsExamined alto señala trabajo descartado; keysExamined alto con docsExamined cero puede ser consulta cubierta pero aun poco selectiva.

> **Precaución técnica.** executionTimeMillis varía por cachés, calentamiento y selección de plan.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «20 documentos devueltos frente a 200000 examinados.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

17. explain: queryPlanner

Explicación, evidencia y criterio técnico

Concepto y mecanismo

queryPlanner describe el plan seleccionado, índices considerados y la estructura general. No ejecuta la consulta completa en el mismo sentido que executionStats.

queryPlanner muestra qué acceso ganaría y bajo qué forma. Úsalo cuando quieras entender la selección de un índice sin confundirlo con una prueba de carga. El plan ganador puede contener campos distintos según el motor y la versión. Explica por qué una elección del optimizador puede ser razonable incluso si existe otra candidata. Su propósito es elegir el coste estimado de un camino, no ratificar el índice que el alumno acaba de crear.

Caso que resuelve

Determinar si existe un IXSCAN y cuál es indexName. Utilizar queryPlanner al estudiar qué plan elegiría MongoDB.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).explain("queryPlanner")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).explain("queryPlanner").queryPlanner.winningPlan
```

Identifica indexName, si aparece, y el alcance del plan elegido. No atribuyas a esta modalidad los contadores de ejecución que pertenecen a executionStats.

Cómo interpretar el resultado

queryPlanner puede mostrar plan candidato sin ejecutar todos los documentos como executionStats; úsalo para estudiar forma de acceso.

> **Precaución técnica.** No confundir winningPlan con todos los planes explorados.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Determinar si existe un IXSCAN y cuál es indexName.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

18. explain: executionStats

Explicación, evidencia y criterio técnico

Concepto y mecanismo

executionStats ejecuta el plan ganador y resume el trabajo. Es la modalidad principal para las comparaciones docentes.

executionStats añade el trabajo efectuado para el plan ganador. Para una comparación A/B conserva el mismo conjunto de datos y ejecuta el mismo filtro con la misma proyección. Registrar solo executionTimeMillis no basta: el caché, la simultaneidad y el sistema pueden cambiar los tiempos. Anotar los tres contadores junto a la estructura ganadora constituye un resultado replicable y técnicamente interpretable. En 8.0 algunas etapas del árbol no tienen el aspecto de manuales antiguos.

Caso que resuelve

Medir claves examinadas y documentos examinados con mismo filtro. Repetir y documentar valores; confirmar tamaño de colección.

Demostración o comprobación

```
db.ventas.find({numero:250001}).explain("executionStats")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).explain("executionStats").executionStats.totalKeysExamined
```

La cantidad de claves recorridas permite distinguir entre un índice consultado de forma selectiva y un recorrido indexado casi total. Guárdala junto a docsExamined.

Cómo interpretar el resultado

El plan ejecutado aporta métricas reales del trabajo durante explain, pero el tiempo incorpora características del experimento y puede variar.

> **Precaución técnica.** El tiempo de explain no equivale a una medida rigurosa de latencia de producción.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Medir claves examinadas y documentos examinados con mismo filtro.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

19. explain: allPlansExecution

Explicación, evidencia y criterio técnico

Concepto y mecanismo

allPlansExecution permite consultar información parcial de planes candidatos durante su selección, útil para explorar optimización.

allPlansExecution añade estadísticas parciales recogidas durante la selección de alternativas. No son benchmarks completos ni autorizan a comparar todas las candidatas como si hubieran procesado la consulta entera. Úsalo para explicar por qué el planificador puede comparar varios accesos y seleccionar uno. En formación introductoria, enséñalo después de queryPlanner y executionStats; no es el punto de partida para descubrir qué es un índice.

Caso que resuelve

Varias rutas de acceso posibles en colección con múltiples índices. Analizar después del plan ganador, sin sobrecargar el nivel introductorio.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).explain("allPlansExecution")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).explain("allPlansExecution").executionStats.allPlansExecution
```

Las estadísticas de selección reflejan pruebas parciales; no representan una ejecución completa por cada plan candidato. En un plan con un único índice quizá no aparezcan múltiples alternativas interesantes.

Cómo interpretar el resultado

En allPlansExecution las estadísticas de alternativas son parciales; comparar sus tiempos como si fueran benchmarks completos sería erróneo.

> **Precaución técnica.** Las estadísticas parciales de candidatos no son ejecuciones completas.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Varias rutas de acceso posibles en colección con múltiples índices.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

20. Uso controlado de hint

Explicación, evidencia y criterio técnico

Concepto y mecanismo

hint fuerza un índice concreto y permite comparaciones experimentales. No debe introducirse por defecto en aplicaciones como solución universal.

hint resulta útil para comparar un candidato con otro sin depender de la decisión del planificador. Solo tiene sentido si el índice existe y el filtro puede interpretarse adecuadamente. Forzar un acceso malo puede elevar muchísimo el trabajo. Enseña a retirar hint antes de trasladar una consulta a código de aplicación: una pista fija que hoy funciona puede bloquear mejoras después de que cambien los datos, el motor o la distribución.

Caso que resuelve

Comparar ix_cliente contra otro índice deliberadamente menos apto. Usar hint como instrumento diagnóstico y retirar tras el análisis.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).hint({clienteId:1}).explain("executionStats")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).hint({clienteId:1}).explain("executionStats")
```

Esta orden fija el camino de acceso y permite contrastarlo con la selección espontánea. Si el optimizador elige otro plan, discute por qué las condiciones de la consulta lo favorecen.

Cómo interpretar el resultado

Forzar clienteId:1 solo tiene sentido cuando la clave existe; explica si el plan impuesto coincide con la elección espontánea del optimizador.

> **Precaución técnica.** Forzar un índice inadecuado puede empeorar tiempos significativamente.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Comparar ix_cliente contra otro índice deliberadamente menos apto.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

21. Índices multikey automáticos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Al indexar arrays MongoDB mantiene entradas para sus elementos. No existe una palabra clave createIndex multikey específica.

El campo etiquetas guarda varios valores; al indexarlo, MongoDB crea entradas asociadas a cada valor distinto del array. Un documento etiquetado como venta, web y urgente puede originar varias claves, pero el resultado lógico de find no debe aparecer triplicado. Observa isMultiKey donde esté disponible. El coste no depende solo de cuántos documentos hay, también de cuántos elementos contienen sus arrays. Una actualización de etiquetas puede modificar varias entradas.

Caso que resuelve

ARRAY → ENTRADAS → DOCUMENTO

Localizar ventas con etiqueta urgente. Inspeccionar isMultiKey o multikeyPaths cuando aparezcan.

Demostración o comprobación

```
db.ventas.createIndex({etiquetas:1},{name:"ix_etiquetas"})
```

Segunda situación para contrastar

```
db.ventas.find({etiquetas:"web"}).limit(10).explain("executionStats")
```

La coincidencia de una etiqueta no debe duplicar una venta aunque el array tenga varias etiquetas. Revisa el documento real y diferencia los elementos del array de las entradas del índice.

Cómo interpretar el resultado

Un índice sobre etiquetas genera entradas para elementos del array; observar isMultiKey y confirmar que una venta coincidente se devuelve una sola vez.

> **Precaución técnica.** Un documento con varias coincidencias no se devuelve varias veces por ellas.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Localizar ventas con etiqueta urgente.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

22. Restricciones multikey

Explicación, evidencia y criterio técnico

Concepto y mecanismo

En índices compuestos existen restricciones cuando se indexan simultáneamente varios campos que contienen arrays del mismo documento.

Un índice compuesto puede combinar un array y un campo escalar, por ejemplo {etiquetas:1,estado:1}. Sin embargo, los índices compuestos multikey tienen restricciones si hay arrays paralelos en varios campos del mismo documento. No construyas un ejercicio que falle sin explicarlo: crea antes un documento con a:[1,2] y b:"X"; después ensaya de forma aislada un segundo documento donde a y b sean arrays y captura el error. La restricción afecta al modelo de documentos admitido por ese índice.

Caso que resuelve

Documento con etiquetas y cupones como arrays separados. Modelar consultas y validar límites antes de diseñar un compuesto.

Demostración o comprobación

```
db.ventas.find({etiquetas:"urgente"}).explain("executionStats")
```

Segunda situación para contrastar

```
db.multikey_demo.insertOne({a:[1,2],b:["X","Y"]})
```

Con un compuesto sobre a y b, este intento puede provocar error de arrays paralelos. Es una prueba negativa de esquema; debe realizarse en colección separada y capturar el error, no romper ventas.

Cómo interpretar el resultado

Un compuesto puede tener un campo array y otros escalares; múltiples campos array del mismo documento pueden activar la limitación parallel arrays.

> **Precaución técnica.** No prometer que cualquier combinación de arrays se puede indexar.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Documento con etiquetas y cupones como arrays separados.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

23. Text index clásico

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice de texto permite búsquedas por términos con $text y $search en despliegues compatibles. No es equivalente a Atlas Search.

El índice text clásico crea una representación de términos, no de cualquier subcadena. Una búsqueda de «bicicleta» sobre descripcion se parece a buscar palabras en un índice invertido. Stemming y reglas lingüísticas pueden variar por default_language; no esperes que coincida siempre un fragmento a mitad de palabra. Una colección autogestionada dispone de las limitaciones propias del índice text; Atlas Search es una tecnología diferente y no se obtiene automáticamente por usar $text.

Caso que resuelve

TÉRMINOS → TEXT → RESULTADOS

Encontrar descripciones que contienen bicicleta o café. En la clase se utiliza texto clásico sobre un MongoDB local.

Demostración o comprobación

```
db.ventas.createIndex({descripcion:"text"},{name:"ix_descripcion_texto",default_language:"spanish"})
```

Segunda situación para contrastar

```
db.ventas.find({$text:{$search:"cafe"}},{_id:0,descripcion:1}).limit(5)
```

La búsqueda por término devuelve descripciones que contienen vocabulario relacionado según el analizador, no todos los textos que incluyen literalmente una cadena de caracteres arbitraria.

Cómo interpretar el resultado

La búsqueda $text requiere un índice de tipo text en esta colección; puede aplicar stemming según idioma y no equivale a buscar subcadenas.

> **Precaución técnica.** Existen límites en idioma, ponderaciones, cantidad de índices y ordenación.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Encontrar descripciones que contienen bicicleta o café.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

24. Relevancia de texto

Explicación, evidencia y criterio técnico

Concepto y mecanismo

textScore aporta una puntuación relativa de relevancia, útil al ordenar resultados de $text en el índice clásico.

textScore ofrece una puntuación de relevancia para coincidencias de $text. No es una probabilidad, ni una distancia, ni equivale a la puntuación de Atlas Search. Si cambias el texto de búsqueda cambias también el contexto en que se interpreta el score. Proyecta la puntuación con {$meta:"textScore"} y ordénala con el mismo metadato. Compara dos descripciones, una con un término y otra con varios, sin prometer un valor numérico concreto.

Caso que resuelve

Ordenar coincidencias por score calculado. Explicar ranking relativo y diferenciarlo del orden lexicográfico.

Demostración o comprobación

```
db.ventas.find({$text:{$search:"bicicleta"}},{score:{$meta:"textScore"}}).sort({score:{$meta:"textScore"}})
```

Segunda situación para contrastar

```
db.ventas.find({$text:{$search:"cafe bicicleta"}},{descripcion:1,p:{$meta:"textScore"}}).sort({p:
{$meta:"textScore"}}).limit(3)
```

La relevancia se interpreta dentro de esta consulta y de estos términos. El score sirve para ordenar, pero no puede expresarse como probabilidad de que el documento sea «correcto».

Cómo interpretar el resultado

El resultado de $meta textScore solo es comparable dentro de la búsqueda realizada; ordenar por score no sustituye un índice ordenado convencional.

> **Precaución técnica.** No esperar prestaciones de motor de búsqueda avanzado.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Ordenar coincidencias por score calculado.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

25. Búsqueda textual: límites

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Stemming, stopwords y opciones de idioma alteran coincidencias. Las búsquedas por subcadenas, fuzzy o autocomplete requieren otras soluciones.

El alumno suele confundir búsqueda de texto con contains. $text trabaja con términos tokenizados y puede manejar variaciones lingüísticas; regex busca patrones y tiene otro perfil de índices; Atlas Search proporciona capacidades más amplias en entornos Atlas. Un índice text no arregla automáticamente $regex no anclados. Diseña una necesidad real —«producto que menciona café»— antes de elegir entre coincidencia textual, búsqueda de prefijo o motor de búsqueda dedicado.

Caso que resuelve

La palabra bicicleta no equivale a un substring arbitrario de una descripción. Plantear un caso sencillo y documentar el idioma elegido.

Demostración o comprobación

```
db.ventas.find({$text:{$search:"cafe"}}).limit(3)
```

Segunda situación para contrastar

```
db.ventas.find({descripcion:/^cafe/}).limit(5)
```

Una expresión regular de prefijo y $text resuelven preguntas distintas. Contrasta también una regex no anclada: tener índice text no constituye garantía de optimización para ese patrón.

Cómo interpretar el resultado

Una cadena buscada como fragmento en medio de una palabra no necesariamente coincide; distinguir tokenización de regex y de Atlas Search.

> **Precaución técnica.** Evitar comparar sin matices con expresiones regulares.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «La palabra bicicleta no equivale a un substring arbitrario de una descripción.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

26. Índices geoespaciales 2dsphere

Explicación, evidencia y criterio técnico

Concepto y mecanismo

2dsphere indexa ubicaciones geográficas sobre GeoJSON y permite proximidad. Las coordenadas GeoJSON son [longitud,latitud].

GeoJSON representa un punto como {type:"Point", coordinates:[longitud,latitud]}. Invertir ese orden es un error frecuente porque muchas interfaces enseñan primero latitud. Un índice 2dsphere interpreta geometría sobre la superficie terrestre y habilita ciertas consultas de proximidad. La validez geográfica y la exactitud de las coordenadas son parte del experimento: no basta con que la estructura JSON sea sintácticamente correcta. Usa puntos sintéticos, no geolocalización privada.

Caso que resuelve

GEOJSON → 2DSPHERE → RADIO

Ventas cercanas a una posición en Madrid. Crear índice 2dsphere y realizar consulta $near con distancia en metros.

Demostración o comprobación

```
db.ventas.createIndex({ubicacion:"2dsphere"},{name:"ix_ubicacion"})
```

Segunda situación para contrastar

```
db.ventas.findOne({}, {ubicacion:1,_id:0})
```

El orden de las coordenadas del punto es [longitud,latitud]. Verificar el dato antes de crear el índice evita atribuir a MongoDB un error que en realidad está en la geometría.

Cómo interpretar el resultado

2dsphere acepta puntos GeoJSON válidos; la latitud y longitud intercambiadas cambian el lugar geográfico, no son un detalle tipográfico.

> **Precaución técnica.** Invertir latitud y longitud es un error frecuente.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Ventas cercanas a una posición en Madrid.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

27. Geoespacial: $near

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Las consultas por proximidad devuelven elementos próximos y requieren el índice geoespacial adecuado.

La búsqueda $near necesita un índice geoespacial compatible. En el ejemplo, el centro es [-3.70,40.42] y $maxDistance=1000 se interpreta como 1.000 metros cuando se usa GeoJSON con 2dsphere. Los resultados se aproximan al centro y pueden ordenarse según proximidad. No compares el tiempo de $near con una colección sin índice que ni siquiera admite correctamente esa operación: es un ejemplo de capacidad habilitada por el índice, no solo de aceleración.

Caso que resuelve

Buscar ventas dentro de 1500 metros del punto de referencia. Comprobar orden y la distancia usada en GeoJSON.

Demostración o comprobación

```
db.ventas.find({ubicacion:{$near:{$geometry:{type:"Point",coordinates:[-3.70,40.42]},$maxDistance:1500}}}).limit(5)
```

Segunda situación para contrastar

```
db.ventas.find({ubicacion:{$near:{$geometry:{type:"Point",coordinates:[-3.70,40.42]},$maxDistance:200}}}).limit(5)
```

Bajar el radio a doscientos metros no cambia la definición del índice. Cambia el conjunto espacial de candidatos y permite discutir unidades y proximidad.

Cómo interpretar el resultado

Con $near sobre GeoJSON la distancia máxima se expresa en metros y el índice 2dsphere es condición necesaria para el operador empleado.

> **Precaución técnica.** No confundir radianes de geometría esférica con metros de $maxDistance en GeoJSON.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Buscar ventas dentro de 1500 metros del punto de referencia.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

28. Hashed: finalidad real

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los hashed indexes almacenan un hash del valor y se usan especialmente en estrategias de distribución por clave hash.

Una clave hashed registra un hash del valor; favorece ciertos escenarios de distribución, especialmente relacionados con sharding. Como el hash destruye el orden natural, no sirve para resolver rangos de idGrupo como lo haría un índice ascendente. Una consulta de igualdad sí puede utilizarlo. No presentes hashed como «índice más rápido» ni como función de seguridad criptográfica de contraseñas: resuelve un problema de distribución y acceso por igualdad.

Caso que resuelve

Distribuir operaciones por idGrupo en una colección fragmentada conceptual. Explicar su pertinencia para igualdad y sharding.

Demostración o comprobación

```
db.ventas.createIndex({idGrupo:"hashed"},{name:"ix_grupo_hash"})
```

Segunda situación para contrastar

```
db.ventas.find({idGrupo:{$gte:100,$lte:200}}).explain("executionStats")
```

El índice hashed sirve para igualdad, pero los hashes de 100 y 200 no delimitan una secuencia ordenada de los valores intermedios.

Cómo interpretar el resultado

El hash dispersa valores con proximidad numérica; una comparación por rango no conserva el orden y no puede sacar la misma ventaja.

> **Precaución técnica.** No utilizarlos esperando acelerar rangos o recuperar orden natural.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Distribuir operaciones por idGrupo en una colección fragmentada conceptual.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

29. Índices únicos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

unique impide valores duplicados de la clave indexada y constituye una restricción de integridad.

unique convierte la integridad de datos en una regla atómica del servidor. Si ya existe codigo F-001, intentar insertar otra factura con el mismo valor debe fallar con E11000. No intentes observar el error insertando datos duplicados en ventas, porque perjudicarías los demás laboratorios. Usa facturas_demo. Antes de construir unique sobre datos existentes hay que detectar duplicados y decidir cómo resolverlos. La unicidad tampoco sustituye una política de autorización.

Caso que resuelve

Dos ventas no pueden compartir numero de pedido. Probar una inserción duplicada y capturar error E11000.

Demostración o comprobación

```
db.ventas.createIndex({numero:1},{name:"ix_numero",unique:true})
```

Segunda situación para contrastar

```
try {db.facturas_demo.insertOne({codigo:"F-001"})} catch(e) {print(e.code)}
```

El conflicto 11000 es intencional si F-001 ya existe bajo la restricción. Si no falla, revisa primero que la colección y el índice unique se prepararon como estaba indicado.

Cómo interpretar el resultado

El intento de crear clave duplicada bajo unique debe producir E11000. El alumno debe comprobar que la segunda operación no insertó el documento.

> **Precaución técnica.** Antes de crearlo, inspeccionar duplicados existentes.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Dos ventas no pueden compartir numero de pedido.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

30. Unique compuesto

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice unique compuesto exige que la combinación de campos sea única, no que cada campo lo sea por separado.

En una colección independiente como inscripciones_unique_demo, el índice {tiendaId:1,codigo:1} unique permite que dos tiendas compartan código si la pareja es distinta. No utilices facturas_demo si ya tiene un índice único sobre codigo aislado. El error emerge al repetir la combinación completa. Explica también que los valores null y los campos ausentes participan en reglas de índices únicos; una colección con muchos documentos sin campo puede impedir crear cierto unique simple. Para opcionalidad, estudiar índices parciales y el diseño de esquema evita expectativas incorrectas.

Caso que resuelve

Combinación {tiendaId,referencia} en facturas. Explicar unicidad de la tupla y qué pasa al variar una columna.

Demostración o comprobación

```
db.facturas.createIndex({tiendaId:1,referencia:1},{unique:true})
```

Segunda situación para contrastar

```
db.inscripciones_unique_demo.createIndex({tiendaId:1,codigo:1},{unique:true})
```

Una clave única de dos campos admite repetir codigo entre tiendas distintas. El alumno debe ensayar dos combinaciones legales y una combinación repetida que se rechace.

Cómo interpretar el resultado

La unicidad de la combinación permite misma referencia en tiendas distintas, pero prohíbe repetir la misma pareja; documentar null y ausencias.

> **Precaución técnica.** Valores ausentes y nulos necesitan diseño explícito.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Combinación {tiendaId,referencia} en facturas.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

31. Sparse y partial

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice parcial solo indexa documentos que cumplen un filtro; sparse omite entradas correspondientes a campos ausentes en ciertos índices.

Un índice partial almacena solo documentos que cumplen partialFilterExpression. Una búsqueda puede aprovecharlo si sus condiciones garantizan que todos los resultados están dentro de esa fracción. Si la consulta no exige estado=pagada, utilizar un índice que solo contiene pagadas perdería datos: el planificador no debe considerarlo suficiente. Sparse es un mecanismo diferente, basado en presencia del campo. Diseña la regla a partir del patrón de consultas, no solo del tamaño deseado.

Caso que resuelve

Indexar solo pedidos activos en lugar de todos los estados. Preferir reglas parciales explícitas cuando el caso lo requiera.

Demostración o comprobación

```
db.ventas.createIndex({clienteId:1},{partialFilterExpression:{estado:"pagada"}})
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809,estado:"pagada"}).explain("executionStats")
```

Para utilizar la versión parcial de un índice por cliente, el predicado debe garantizar pertenencia al subconjunto pagado. El planificador evita resultados incompletos.

Cómo interpretar el resultado

El plan puede emplear el índice parcial si la consulta restringe estado=pagada; si omite estado, ese índice no representa el conjunto completo.

> **Precaución técnica.** Una consulta que no implica el filtro parcial puede no usar el índice.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Indexar solo pedidos activos en lugar de todos los estados.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

32. TTL: caducidad diferida

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un índice TTL borra documentos según campo Date y expireAfterSeconds, con un monitor periódico. No elimina instantáneamente al alcanzar el plazo.

TTL automatiza eliminación eventual de documentos expirados, como tokens o sesiones de demostración. No es una agenda exacta al segundo: un monitor en segundo plano selecciona documentos elegibles y puede retrasarse. Un índice TTL convencional es de un solo campo sobre fechas BSON; no utilices strings que parezcan fechas. Nunca apliques TTL accidentalmente a ventas que deban conservarse por obligación legal. La política de retención es una decisión de negocio y de cumplimiento, no un truco de rendimiento.

Caso que resuelve

FECHA → MONITOR TTL → BORRADO

Sesiones temporales o tokens ficticios con fecha de expiración. Usar colección independiente para no perder datos de ventas.

Demostración o comprobación

```
db.sesiones.createIndex({creadoEn:1},{expireAfterSeconds:60})
```

Segunda situación para contrastar

```
db.sesiones_demo.getIndexes()
```

Comprueba que expireAfterSeconds pertenece al índice apropiado y que el campo almacena fechas. La eliminación es por caducidad, no una orden directa de deleteMany.

Cómo interpretar el resultado

El monitor TTL elimina documentos elegibles en segundo plano. La observación correcta es eventual, no una promesa de borrado al cumplir exacto sesenta segundos.

> **Precaución técnica.** Nunca enseñar TTL como un cronómetro de precisión al segundo.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Sesiones temporales o tokens ficticios con fecha de expiración.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

33. TTL: campo y cálculo

Explicación, evidencia y criterio técnico

Concepto y mecanismo

expireAfterSeconds cuenta desde una fecha almacenada, salvo patrón de fecha de expiración absoluta con cero segundos.

expireAfterSeconds:60 se interpreta con respecto al valor del campo Date: un documento creado hace dos minutos ya es elegible, uno creado ahora no. El monitor puede tardar en ejecutar el borrado. Si se usa expireAfterSeconds:0, el propio campo suele representar el instante de expiración. La diferencia es crucial: no mezclar el campo fechaDeCreacion con fechaDeCaducidad sin ajustar el valor. Una consulta countDocuments demuestra el estado observado, no el instante exacto del monitor.

Caso que resuelve

Una sesión creada hace dos minutos ya es elegible para caducar. Crear fechas Date reales y verificar tras esperar el ciclo TTL.

Demostración o comprobación

```
db.sesiones.insertOne({usuario:"demo",creadoEn:new Date(Date.now()-120000)})
```

Segunda situación para contrastar

```
db.sesiones_demo.find({}, {_id:0,creadoEn:1,tipo:1}).toArray()
```

Distingue elegibilidad temporal de ejecución efectiva del monitor. La misma observación repetida más tarde constituye la prueba razonable de TTL.

Cómo interpretar el resultado

Una fecha dos minutos anterior con TTL de un minuto ya cumple el criterio; una fecha actual aún no, aunque ambas existan al insertar.

> **Precaución técnica.** Cadenas ISO guardadas como strings no equivalen a fechas BSON.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Una sesión creada hace dos minutos ya es elegible para caducar.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

34. Ocultar y eliminar índices

Explicación, evidencia y criterio técnico

Concepto y mecanismo

hideIndex permite excluir temporalmente un índice de la planificación; dropIndex lo elimina. Son herramientas de mantenimiento.

hideIndex oculta un índice al planificador sin retirarlo físicamente. Permite estudiar el efecto que tendría su ausencia y después revertir la prueba; mientras esté oculto sigue consumiendo espacio y trabajo en escrituras. dropIndex sí elimina el índice. Evita modificar *id* y documenta la reversión. Esta herramienta es útil para retirada prudente, pero la prueba de un día no captura consultas mensuales o cierres contables.

Caso que resuelve

Verificar si un índice redundante es necesario antes de borrarlo. Ocultar, observar los planes y restaurar o eliminar con criterio.

Demostración o comprobación

```
db.ventas.hideIndex("ix_cliente")
```

Segunda situación para contrastar

```
db.ventas.getIndexes().filter(x=>x.hidden)
```

Esta consulta permite saber qué índice fue ocultado. Restaurarlo antes de continuar evita comparar planes de un estado desconocido.

Cómo interpretar el resultado

hideIndex vuelve invisible el índice al planificador pero mantiene su actualización. Después de la prueba conviene restaurarlo con unhideIndex.

> **Precaución técnica.** Ocultar no recupera el coste de mantenimiento de escritura.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Verificar si un índice redundante es necesario antes de borrarlo.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

35. Observabilidad del ciclo de vida

Explicación, evidencia y criterio técnico

Concepto y mecanismo

getIndexes y $indexStats ayudan a inventariar índices y uso registrado en el servidor. Una medición breve no representa toda la carga.

$indexStats informa de accesos observados desde un período de medición, no de la utilidad eterna del índice. Una baja cifra puede deberse a que la función que lo utiliza solo corre una vez al mes. Complementa uso, patrones de consultas, tamaños e impacto en escrituras. Para un inventario técnico, anota nombre, clave, opciones, consultas servidas y motivo de conservación. La retirada de índices debe planificarse y monitorizarse.

Caso que resuelve

Comprobar si un índice se utiliza durante varios escenarios. Recoger tráfico representativo y consultar operaciones por índice.

Demostración o comprobación

```
db.ventas.aggregate([{$indexStats:{}}]).toArray()
```

Segunda situación para contrastar

```
db.ventas.aggregate([{$indexStats:{}}]).toArray()
```

Los contadores solo describen la ventana observada; el índice de una tarea trimestral puede no tener accesos durante un curso de tres horas.

Cómo interpretar el resultado

$indexStats cuenta accesos desde su ventana de registro; un índice marcado poco usado durante diez minutos puede ser esencial en el cierre mensual.

> **Precaución técnica.** Reinicios y periodos cortos pueden invalidar conclusiones.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Comprobar si un índice se utiliza durante varios escenarios.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

36. Índices y tamaño de memoria

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los índices compiten por memoria y almacenamiento; los beneficios dependen del working set y del patrón real de acceso.

La memoria cache almacena páginas de datos e índices; cuanto más voluminosas sean las estructuras activas, mayor puede ser la presión sobre recursos. indexSizes y stats aportan tamaños físicos, pero compresión y patrones de acceso impiden inferir linealmente el rendimiento. Un índice parcial puede ser útil si su subconjunto representa una parte pequeña de datos. La pregunta no es «¿cuántos índices puedo crear?», sino «¿qué conjunto mínimo justifica cada coste?».

Caso que resuelve

Un índice que cabe en caché produce comportamiento diferente a uno muy grande. Documentar tamaño de colección, hardware y condiciones de la prueba.

Demostración o comprobación

```
db.ventas.stats()
```

Segunda situación para contrastar

```
db.ventas.stats().indexSizes
```

El coste de bytes se pone en relación con los patrones de consulta y con el costo de mantener cada índice en escrituras.

Cómo interpretar el resultado

stats e indexSizes permiten estudiar memoria y espacio, pero compresión y working set impiden convertir cada byte en una predicción de latencia.

> **Precaución técnica.** No atribuir diferencias de milisegundos solo a diseño lógico.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Un índice que cabe en caché produce comportamiento diferente a uno muy grande.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

37. Medición experimental honesta

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Una prueba rigurosa controla volumen, consultas, proyección, índices existentes y repetición. Plan, throughput y latencia son dimensiones distintas.

Diseña comparaciones controladas: mismo volumen, mismo comando, igual proyección, sin cargas concurrentes y varias ejecuciones. Distingue medición de laboratorio y SLO de producción. Primero verifica cambio de estructura en explain y reducción de examinados; después mide latencia con suficiente repetición y variabilidad. Si una consulta pasó a IXSCAN pero aumentó claves examinadas, no afirmes automáticamente que mejoró. Presenta incertidumbre y características del hardware.

Caso que resuelve

Tres repeticiones de la misma consulta con evidencia. Primero confirmar IXSCAN/COLLSCAN; después comparar órdenes de magnitud.

Demostración o comprobación

```
db.ventas.find({clienteId:809}).explain("executionStats")
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809}).explain("executionStats").executionStats.executionTimeMillis
```

Repetir la medición en frío y en caliente ayuda a ilustrar el efecto de caché, pero no convierte un único milisegundo en una comparación estadística ni sustituye la lectura de los documentos examinados.

Cómo interpretar el resultado

Repetir la consulta y controlar volumen distingue una mejora estructural de un simple efecto de caché, plan calentado o carga concurrente.

> **Precaución técnica.** No afirmar aceleraciones fijas en distintos equipos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Tres repeticiones de la misma consulta con evidencia.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

38. Diseño final: cobertura mínima

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La selección final de índices es un compromiso. Una sola solución no puede optimizar simultáneamente cualquier filtro, sort y actualización.

La propuesta final conecta consultas frecuentes con índices mantenidos. Ejemplo: cliente → {clienteId:1}; pagadas por fecha → {estado:1,fecha:-1}; búsqueda textual → {descripcion:"text"}. Justifica por qué otras necesidades son excepcionales o se atienden de otra forma. Añadir índices sin uso aumenta superficie de mantenimiento. El alumno debe defender tanto lo que crea como lo que decide no crear; es una decisión de arquitectura de acceso a datos.

Caso que resuelve

Aplicación con consulta por cliente, ranking por fecha y búsqueda por texto. Elegir un pequeño inventario defendible y registrar qué consulta cubre cada índice.

Demostración o comprobación

```
db.ventas.getIndexes().map(x=>({nombre:x.name,claves:x.key}))
```

Segunda situación para contrastar

```
db.ventas.find({clienteId:809,estado:"pagada"}).sort({fecha:-1}).limit(10).explain("executionStats")
```

En el diagnóstico final el alumno debe conservar una tabla con dos diseños alternativos y defender uno; no basta con afirmar que la consulta ahora «va más rápida».

Cómo interpretar el resultado

La solución final debe mapear cada índice conservado a uno o varios patrones frecuentes y justificar índices descartados por coste de escritura.

> **Precaución técnica.** Más índices no significa un sistema mejor.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Aplicación con consulta por cliente, ranking por fecha y búsqueda por texto.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.